Penetration Testing Report

Maria Vaikath(mav218) & Rithika Pathuri (rrp149)

1. Executive Summary

This report presents the results of a security assessment of the application, covering authentication, authorization, input validation, session management, and configuration security. The testing focused on common web application risks such as brute force attacks, privilege escalation, insecure file uploads, and session-based attacks. Overall, the application has strong security controls in place, especially for authentication and access control. Most high-risk issues are properly handled through server-side checks and validation. However, session management was identified as an area that could be improved, mainly around session hijacking protection.

2. Testing Methodology

The testing used a mix of black-box and grey-box approaches. Endpoints were tested using both the browser and curl commands to simulate real attack scenarios. Each area was tested using:

- Manual input testing (malformed or injected data)
- Repeated request testing (brute force and rate limiting)
- Cross-session testing using exported cookies
- Direct endpoint access without using the frontend
- File upload testing with malicious files and filenames
- Session reuse across different terminals

Results were compared against expected secure behaviour to identify vulnerabilities or confirm protections.

3. Findings

A. Authentication Testing

☑ Brute force protection

i. Brute force for one account

- **Test:** Attempting to login to the same account using incorrect passwords repeatedly
- **Expected:** The application should prevent more than 5 continuous incorrect login attempts.
- **Actual**: After 5 failed login attempts, the account is locked for 15 minutes and the user is prevented from logging in.
- **Finding:** The backend increments *failed_attempts* for each user on each failed login and sets *locked_until* after 5 attempts. Subsequent login attempts check this value and return a 401 error until the lockout period expires.

ii. Brute force - rate limiting based on IP

- **Test:** Sending more than 10 login requests within one minute from the same IP.
- **Expected:** The application should block excessive login attempts.
- **Actual**: Requests beyond the limit are rejected with an error message.
- **Finding:** The *rateLimitChecker()* function tracks timestamps per IP and rejects requests exceeding 10 attempts within 60 seconds, returning a 401 response.

☑ Password complexity enforcement

- **Test:** Attempting to register with a weak password such as "password123".
- **Expected:** The application should reject passwords that do not meet complexity requirements.
- **Actual**: The registration request is rejected with an error stating the password does not meet requirements.
- **Finding:** The backend validatePassword() function rejected the input because it lacked special characters and numbers. A 400 error was returned.

☑ Session management

- **Test:** Logging in and inspecting the session token stored in browser cookies.
- **Expected:** A secure session token should be generated and stored with appropriate security flags.
- **Actual**: A session token is generated and stored in a cookie with HttpOnly, Secure, and SameSite=Strict attributes.
- **Finding:** The backend uses secrets.token_urlsafe(32) in *createSession()* to generate a cryptographically secure token, which is stored server-side and issued via a secure cookie configuration.

☑ Logout functionality

- **Test:** Logging in, then logging out, and attempting to access the dashboard route of the previously logged in user.
- **Expected:** The session should be invalidated and access should be denied.
- **Actual**: After logout, accessing protected routes returns an authentication error.
- **Finding:** The backend calls destroySession() to remove the session token from storage and clears the cookie, preventing reuse of the session.

☑ Password reset security

- **Test:** Attempting to change the account password through the dashboard by submitting the old password along with a new password.
- **Expected:** The application should require the correct current password and enforce password complexity rules before allowing the change.
- **Actual**: The password change request requires the correct old password and validates the new password against complexity requirements. If the old password is incorrect or the new password is weak, the request is rejected. The new password must also be confirmed (typed twice) before allowing the change.
- **Finding:** The backend verifies the current password before allowing updates and applies validatePassword() to the new password, ensuring only authenticated users can change their credentials and that new passwords meet security requirements.

B. Authorization Testing

☑ Horizontal privilege escalation

- **Test:**

  Log in as User A and upload a document, find and keep track of that DOC_ID. Then log in as User B using the curl commands in the terminal and try to download User A's document:

  Login:

  curl -k -c cookiesB.txt -X POST https://localhost:5001/login \
-H "Content-Type: application/json" \
-d '{"username":"user_name","password":"user_password!"}'

```
{"message":"Successful login!","role":"user","success":true}
```

Attempt to download:

```
curl -k -b cookiesB.txt https://localhost:5001/download/DOC_ID
```

```
{"error":"You do not have permission to view this document."}
```

- **Expected:** User B should not be able to access User A's document unless explicit sharing permissions have been granted.
- **Actual**: The request is denied with a 403 error and the message: "You do not have permission to view this document."
- **Finding:** The @requireDocumentPermission decorator checks ownership and sharing permissions before allowing access. Since User B is not authorized, the request is blocked and logged.

☑ Vertical privilege escalation

- **Test:** Attempting to escalate privileges from a standard user to an admin by directly invoking a protected endpoint (/upgradeRole) using both browser console and curl.

```
fetch('/upgradeRole', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    username: 'user_name',
    role: 'admin'
  })
}).then(res => res.json()).then(console.log)
```

```
curl -k -b cookiesB.txt -X POST https://localhost:5001/upgradeRole \
  -H "Content-Type: application/json" \
  -d '{"username":"user_name","role":"admin"}'
```

```
{"error":"Permissions have not been granted."}
```

- **Expected:** Only users with admin privileges should be allowed to modify user roles. All other requests should be rejected.
- **Actual**: The request is denied with a 403 error and the message: “Permissions have not been granted.”
- **Finding:** The @requireRole(ROLE_ADMIN) decorator prevents non-admin users from accessing this endpoint. Role checks are enforced server-side, so modifying the request does not bypass security.

☑ Insecure Direct object reference (IDOR)

- **Test:** Attempting to access a document using a known or guessed DOC_ID belonging to another user.

  curl -k -b cookiesB.txt https://localhost:5001/download/DOC_ID

  ```
  {"error":"You do not have permission to view this document."}
  ```

- **Expected:** Unauthorized users should not be able to access resources simply by knowing or guessing their identifiers.
- **Actual**:The request is denied with a 403 error indicating lack of permission.
- **Finding:** Although document identifiers are exposed in requests, access is protected by server-side authorization checks. Additionally, the use of non-sequential identifiers (e.g., UUIDs) reduces the likelihood of successful guessing attacks.

☑ Forced browsing

- **Test:** Attempting to directly access restricted administrative endpoints without proper authorization by manually issuing requests.

  curl -k -b cookiesB.txt https://localhost:5001/findUsersList

  ```
  {"error":"Permissions have not been granted."}
  ```

- **Expected:** Non-admin users should be prevented from accessing admin-only routes regardless of how the request is made.
- **Actual**: The request is rejected with a 403 error and a permissions error message.
- **Finding:** The @requireRole(ROLE_ADMIN) decorator correctly restricts access to admin-only functionality. Even when bypassing the UI and directly invoking endpoints, access restrictions remain enforced.

C. Input Validation Testing

☑ XSS attempts

- **Test:** Attempting to inject a malicious script into user input fields during registration in order to test whether the application properly sanitizes or rejects potentially dangerous HTML/JavaScript content. The payload includes a <script> tag designed to execute JavaScript if rendered.

  curl -k -X POST https://localhost:5001/register \
  -H "Content-Type: application/json" \
  -d
  '{"username":"<script>alert(1)</script>","email":"testemail@gmail.com","password":"ValidPasswordYes123!","confirm_password":"ValidPasswordYes123!"}'

  ```
  {"error":"Invalid username"}
  ```

- **Expected:** The application should either sanitize the input or reject it completely so that no script content is stored or rendered.
- **Actual**: The request was rejected with the error message: “Invalid username.”
- **Finding:** The username validation is strict and only allows alphanumeric characters and underscores, so the script never makes it through. This blocks XSS at the input level instead of trying to clean it later.

☑ Path traversal

- **Test:** Attempting to upload a file with a malicious filename that includes directory traversal sequences (../) in order to escape the intended upload directory and potentially access restricted system paths.

  Login:

  curl -k -c cookiesB.txt -X POST https://localhost:5001/login \
-H "Content-Type: application/json" \
-d '{"username":"user_name","password":"user_password!"}'

  ```
  {"message":"Successful login!","role":"user","success":true}
  ```

  Create temporary malicious file:

echo "hello world" > test.txt

Upload File:

curl -k -b cookiesB.txt -X POST https://localhost:5001/upload \
-F "file=@test.txt;filename=../../etc/passwd.txt"

```
{"fileName":"etc_passwd.txt","success":true}
```

- **Expected:** The application should sanitize the filename and prevent directory traversal, ensuring the file is stored only within the designated upload directory.
- **Actual**: The file upload request succeeds, but the filename is sanitized and does not retain the ../ traversal sequences.
- **Finding:** The application uses secure filename handling (such as secure_filename()), which strips out path traversal characters and ensures that uploaded files cannot escape the intended directory. This prevents attackers from accessing or overwriting sensitive system files.

☑ Command injection

- **Test:** Attempting to inject system-level commands into user input fields during registration. Payloads such as ; ls and $(whoami) are used to test whether input is executed by the system shell.

curl -k -X POST https://localhost:5001/register \
-H "Content-Type: application/json" \
-d '{"username":"test;
ls","email":"test@gmail.com","password":"ValidPassword123!","confirm_password":"ValidPass
word123!"}'

```
{"error":"Invalid username"}
```

curl -k -X POST https://localhost:5001/register \
-H "Content-Type: application/json" \
-d
'{"username":"$(whoami)","email":"test@gmail.com","password":"ValidPassword123!","confir
m_password":"ValidPassword123!"}'

```
{"error":"Invalid username"}
```

- **Expected:** The application should treat this input as plain text and not execute anything. Ideally, it should also reject clearly invalid usernames.

- **Actual**: Both requests were rejected with “Invalid username.”
- **Finding:** Since the app doesn’t pass user input into system commands and also validates usernames strictly, there is no opportunity for command injection here and the attack fails at the validation step.

☑ File upload vulnerabilities

- **Test:** Testing file upload validation by attempting to upload potentially malicious files, including executable scripts and files with spoofed extensions.

  Create malicious file:

  ```
  echo "<?php echo 'hacked'; ?>" > shell.php

  curl -k -b cookiesB.txt -X POST https://localhost:5001/upload \
  -F "file=@shell.php"
  ```

  ```
  {"error":"File type not allowed"}
  ```

  File extension spoofing:

  ```
  echo "hello" > test.txt

  curl -k -b cookiesB.txt -X POST https://localhost:5001/upload \
  -F "file=@test.txt;filename=shell.php"
  ```

  ```
  {"error":"File type not allowed"}
  ```

- **Expected:** The application should restrict uploads to safe file types and reject files with dangerous extensions such as .php, .exe, or other executable formats.
- **Actual**: The upload request for shell.php is rejected with the error "File type not allowed". The spoofed filename attempt is also rejected or sanitized depending on implementation.
- **Finding:** The application enforces file type validation and prevents the upload of potentially dangerous files. This mitigates risks such as remote code execution and server compromise through malicious file uploads.

D. Session Security Testing

☑ Session fixation

- **Test:** Attempting to perform a session fixation attack by supplying a known or attacker-controlled session token prior to authentication and observing whether it is reused after login.

  Invalid session token was used to attempt access:

  curl -k https://localhost:5001/dashboard \
--cookie "session_token=fakesessiontoken"

```
{"error":"Please login."}
```

  Normal login to generate valid session:

  curl -k -c cookiesB.txt -X POST https://localhost:5001/login \
-H "Content-Type: application/json" \
-d '{"username":"user_name","password":"user_password!"}'

```
{"message":"Successful login!","role":"user","success":true}
```

  cat cookiesB.txt

```
# Netscape HTTP Cookie File
# https://curl.se/docs/http-cookies.html
# This file was generated by libcurl! Edit at your own risk.

#HttpOnly_localhost     FALSE   /       TRUE    1776788422      session_to
ken     TjeGTbKDDnEQ9KkeWj09CXQa4piTlNx7k0pP8WuekNg
```

- **Expected:** The application should ignore any pre-existing session identifiers and generate a new session token upon successful login.
- **Actual**: The fake session token did not grant access, and a new session token was generated after login.
- **Finding:** The application mitigates session fixation attacks by issuing fresh session identifiers upon authentication using a secure random generator.

☐ Session hijacking

- **Test:** A valid session was generated through a normal login and the session cookie was exported using curl. The same token was then reused from a separate terminal without re-authentication to simulate a stolen session scenario.

  Login to generate valid session token:

  curl -k -c cookiesB.txt -X POST https://localhost:5001/login \
-H "Content-Type: application/json" \

-d '{"username":"user_name","password":"user_password!"}'

```
{"message":"Successful login!","role":"user","success":true}
```

cat cookiesB.txt

```
# Netscape HTTP Cookie File
# https://curl.se/docs/http-cookies.html
# This file was generated by libcurl! Edit at your own risk.

#HttpOnly_localhost     FALSE   /       TRUE    1776795567     s
ession_token    rUoy9rk6ecvT4jT8hrOYQN2237d7FMjfXan79SrhkyM
```

curl -k -b cookiesB.txt https://localhost:5001/findUserFileList

curl -k https://localhost:5001/dashboard \
--cookie "session_token=*rUoy9rk6ecvT4jT8hrOYQN2237d7FMjfXan79SrhkyM*"

returns userFileList and dashboard html → works

- **Expected:** Session tokens should not be sufficient on their own for full authentication if additional binding mechanisms (e.g. IP or user-agent validation) are enforced.
- **Actual**: The reused session token remained valid and granted access to protected routes as long as it had not expired.
- **Already Implemented Controls:**
    - Sessions are stored server-side and expire after 30 minutes of inactivity
    - Cookies are set with HttpOnly, Secure, and SameSite=Strict
    - Logout explicitly destroys the session token
- **Remediation Steps:**
    - IP and user-agent checks have been added to validateSession(), however they currently function as detection and logging mechanisms rather than strict enforcement across all flows. This means session reuse is still possible if a valid token is obtained.
- **Finding:** IP/User-Agent checks are implemented as a detection mechanism (logging + rejection when mismatched), but they are not consistently enforced as a strict session invalidation strategy across all scenarios. Therefore, session replay is still possible if the attacker matches the original environment closely (similar IP) or if the token is reused within valid constraints.

☑ Session timeout

- **Test:** Test: Logging in and attempting to access a protected route after the session timeout period has elapsed.

  curl -k -b cookiesB.txt https://localhost:5001/dashboard

  ```
  {"error":"Please login."}
  ```

- **Expected:** Sessions should expire after the configured inactivity period and require re-authentication.
- **Actual**: The request is denied after the timeout period, indicating that the session has expired, and the user is prompted to login again.
- **Finding:** Session expiration is correctly enforced by using validateSession() to check the last activity timestamp and invalidating sessions that exceed the allowed duration.

☐ Concurrent Sessions

- **Test:** Logging into the same account from multiple browsers or devices simultaneously to observe whether multiple active sessions are allowed.
- **Expected:** The system should ideally restrict multiple sessions of the same account occurring at the same time.
- **Actual**: Multiple sessions remain active concurrently and function independently.
- **Finding:** The application allows concurrent sessions for the same user account. While not inherently insecure, this increases the attack surface in cases of credential compromise. A stricter implementation could invalidate previous sessions upon new login.

E. Configuration Testing

☑ TLS/SSL configuration

- **Test:** Make sure that all communication with the application is encrypted and that HTTP requests are not served directly. The http and https links are tested below.

  curl -I http://localhost:5000

```
HTTP/1.1 301 MOVED PERMANENTLY
Server: Werkzeug/3.1.3 Python/3.12.3
Date: Tue, 21 Apr 2026 15:39:40 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 233
Location: https://localhost:5001/
Connection: close
```

curl -k -I https://localhost:5001

```
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.12.3
Date: Tue, 21 Apr 2026 15:41:37 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 2624
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inl
ine';style-src 'self'; img-src 'self' data:; frame-ancestors 'none';
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=()
Strict-Transport-Security: max-age=31536000; includeSubDomains
Connection: close
```

- **Expected:** The server should not return any content over HTTP and should redirect the request to the HTTPS version of the site. The HTTPS request should be connected successfully.
- **Actual**: The server returned a 301 redirect, pointing to the HTTPS endpoint on port 5001. This shows that HTTP requests are not processed directly and are instead forwarded to a secure connection, and the HTTPS requests are connected successfully.
- **Finding:** The application enforces HTTPS by running an HTTP service on port 5000 that redirects all requests to the HTTPS service on port 5001. This ensures that all data sent between the client and server is encrypted.

☑ Error handling

- **Test:** An invalid file upload request was submitted with an empty file field to evaluate how the system handles malformed input.

curl -k -c cookiesB.txt -X POST https://localhost:5001/login \
-H "Content-Type: application/json" \
-d '{"username":"user_name","password":"user_password!"}'

```
{"message":"Successful login!","role":"user","success":true}
```

curl -k -b cookiesB.txt -X POST https://localhost:5001/upload -F "file="

```
{"error":"No file found"}
```

- **Expected:** The application should return a generic error message without exposing stack traces or internal system details.
- **Actual**: The application returns a generic error response “no selected file.” In cases where no session was present, the request was blocked earlier in the request pipeline with a "Please login" response.
- **Finding:** Invalid input is handled safely as authentication is enforced before request processing, preventing unauthenticated users from reaching deeper logic layers. Error messages are generic and do not reveal internal file paths, stack traces, or system configuration details.

☑ Debug mode disabled

- **Test:** Verifying that the application is not running in debug mode by inspecting runtime behavior and configuration.
- **Expected:** Debug mode should be disabled in production to prevent exposure of stack traces, environment variables, and interactive debugging consoles.
- **Actual**: The application runs with debug mode disabled (debug=False), and no debug interface is accessible.
- **Finding:** The application is correctly configured for a production environment, reducing the risk of sensitive information disclosure through debugging tools.

4. Conclusion

Overall, the application is secure against most of the common vulnerabilities tested. Authentication, authorization, and input validation are all well implemented, and most attack attempts were properly blocked. Session management is mostly secure, with proper session timeouts, secure cookies, and server-side session storage. However, session hijacking is still possible if a valid session token is stolen, since token reuse is allowed until expiry. This is not a critical issue, but it is a clear area for improvement. Other areas such as file uploads, error handling, and role-based access control are implemented correctly and prevent common attacks.